

布隆过滤器

布隆过滤器（Bloom Filter）是一种用于判断元素是否属于某个集合的概率数据结构，由伯顿·霍华德·布隆（Burton Howard Bloom）于 1970 年提出。[Bloom, 1970] 它通过一系列散列函数，把元素映射到二进制位数组中，以较小的存储空间和较快的查询速度完成判断。不过，这种判断可能产生假阳性（误报），不会产生假阴性（漏报），而且删除元素比较困难。

判断一个元素是否在集合中，通常可以先保存集合中的所有元素，再通过比较来查找。链表、树和散列表都采用这种思路；散列表也称哈希表，它们的检索时间复杂度分别为 $O(n)$ 、 $O(\log n)$ 和 $O(1)$ 。随着集合扩大，存储需求会增加，检索速度也会受到影响。布隆过滤器不保存元素本身，而是保存元素经过散列映射后留下的位标记，因此空间效率和查询时间效率上都比一般算法有很大优势。

一个空的布隆过滤器由长度为 m 的位数组和 k 个不同的散列函数组成，数组中的所有位初始都为 0。每个散列函数都把元素映射到 m 个数组位置中的一个。为了取得最佳效果，这些函数应当均匀分布且相互独立。通常， k 是一个较小的常数，取决于期望的假阳性率 ε ； m 则与 k 和计划添加的元素数量成正比。

添加元素时，分别用这 k 个散列函数计算它的位置，再把所得位置的位全部设为 1。这里的函数数量也可记作 K ，与 k 含义相同。查询时，对待查元素做同样的计算，再检查这些位置：只要有一位为 0，就可以确定该元素不在集合中，因为已经添加过的元素会把对应位置全部设为 1；如果所有位置都为 1，就只能判断它可能在集合中。这些 1 也可能是添加其他元素时碰巧设置的，从而造成假阳性。简单的布隆过滤器无法区分这两种情况，更先进的技术则可以解决这一问题。

以集合 $\{x, y, z\}$ 为例，设位数组长度 $m = 18$ ，散列函数数量 $k = 3$ 。用彩色箭头表示各元素经过散列后映射到的数组位置时，箭头的数量对应散列函数数量 k 。对于不在该集合中的元素 w ，它映射到的位置中有一位为 0，因此查询可以确定它不属于集合 $\{x, y, z\}$ 。

当 k 较大时，设计这么多个彼此独立的散列函数可能很困难。一种办法是使用输出较宽的良好散列函数：它的不同位域之间应当几乎没有相关性，因此可以把输出切分成多个位域，用来生成多个“不同”的散列函数。另一种办法是给接受初始值的散列函数传入 k 个不同的初始值，例如 $0, 1, \dots, k - 1$ ；也可以把这些值加入或追加到键中。

当 m 、 k 中的一个或两个较大时，可以放宽散列函数之间的独立性要求，由此增加的假阳性率可以忽略不计。[Dillinger 与 Manolios, 2004a; Kirsch 与 Mitzenmacher, 2006] 增强双重散列和三重散列也能有效地导出 k 个索引。[Dillinger 与 Manolios, 2004b] 这两种方法都是双散列的变体，实质上是用两个或三个散列值作为种子的简单随机数生成器。

简单的布隆过滤器无法直接移除元素，因为它无法判断该元素对应的 k 个位中，哪些位可以安全清除。把其中任意一位设为 0，虽然足以让这个元素不再被查询命中，却也会让恰好映射到该位的其他元素无法被命中。简单算法没有办法确定是否还有其他已添加的元素使用了这些位，所以清除任何一位都可能引入假阴性。

可以用一个辅助布隆过滤器记录已移除的元素，称为“移除过滤器”，以模拟一次性的删除操作。但移除过滤器自身也可能误报：它的假阳性会转化为原过滤器与移除过滤器组成的复合过滤器中的假阴性。这种做法也无法重新添加先前移除的元素，因为重新添加还需要先把该元素从移除过滤器中删除，而这又回到了简单布隆过滤器无法安全删除元素的问题。

布隆过滤器在空间和时间方面的优势很大，其存储空间和插入、查询时间具有常数开销 ($O(k)$)。各散列函数之间没有相互依赖，也便于用硬件并行实现。由于不需要保存元素本身，它在一些保密要求非常严格的场合也有优势。

这些优势伴随着误报的代价：存入的元素越多，假阳性率就越高。通常，所有待过滤的键都可以获取，但逐一枚举它们可能很昂贵，例如需要多次读取硬盘。如果假阳性率过高，可以重新生成过滤器，不过这类重建应当相对少见。元素数量很少时，使用散列表就足够了。为了降低假阳性率，已有不少研究提出了布隆过滤器的各种变体。

出处：[维基百科“布隆过滤器”](#)，CC BY-SA。所用摘录获取于 2026-09-10。